claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-a7b979af0e35bab63.jsonl`
- SHA-256: `d9af1f1c394e8535dd3bbe50782c3c90113b17018e9d0793ed9470b09dc4debe`
- Source modified: `2026-07-02T02:52:10+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_41c8dc8a-8a4`
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`

Transcript

1. user (2026-07-02T02:47:11.982Z)

CONTEXT (read carefully):
- You are one auditor in a multi-agent tech-debt/critical-fix audit of the khelsutra-guru multi-repo workspace at C:/Users/avidu/Projects/khelsutra-guru. Today is 2026-07-02. All git repos were just pulled to current origin master/main.
- Repos: badminton-highlight-indexer (the ENGINE, Python, ~1500 tests, org Khelsutra), khelsutra (SPA web/ + marketing site/, TS/React, org avidullu), sports-data-collector, rally-annotator, sports-obsreport (small Python repos). Plus two NON-git dirs: "Annotation Setup", khelsutra-screencast.
- HARD RULES: READ-ONLY. Do NOT modify/create/delete any file, do NOT create branches/worktrees, do NOT pip install, do NOT run the full test suite (CI is trusted on master). You MAY run read-only git and gh commands (gh is authenticated), ruff/mypy IF already installed, and fast read-only scripts.
- EVERY claim must cite file:line (or a gh URL). A claim without file:line evidence is a hypothesis - mark it as such or drop it. The codebase drifts between sessions: re-verify anything you believe from docs against actual code.
- OWNER-GATED areas (flag as owner_gated=true, still report): alpha/serving go-live + Cloudflare dashboard work, Studio P10/P11 (corpus promotion + train/reeval), product/strategy/licensing decisions, real GPU/cloud spend, counsel ToS/DPA, repo rename off "badminton".
- Known-open items from project memory (VERIFY current state, don't trust blindly): engine PR #390 (D1 split main.py, open since 06-26) · audit doc docs/CODE_CLEANUP_AUDIT_2026-06-24.md remaining items D1, D13 (ByteTrack->trackers), Phase-3 (B3/B4/B6/B7/B8/D5/D6/D8-D10/O3/O4) · docs/DECISION_SNAPSHOT_REGRESSION.md next=P1 · docs/GOLDEN_REGRESSION_FIXTURES.md backlog M4/P0-rename/O1/O2 · engine issues #340 (Gemini re-bill), #332 (NSFW preflight), #349 (CPU-bound decode), #384 (CI single-source) · khelsutra issues #97-#114 UX/alpha backlog · DO droplet RETIRED 2026-06-25 (docs may still reference it; api.khelsutra.guru CNAME/CF-Access cleanup was pending).
- Categorize each finding: critical-fix (latent bug/correctness/data-loss/cost-leak), security, tech-debt (structure/duplication/dead code), test-gap, ci-infra, doc-rot, hygiene.
- Severity: P0 = latent bug or cost/data/security risk reachable in normal use; P1 = high-value debt blocking velocity or a stale-open thread; P2 = worthwhile cleanup; P3 = nice-to-have (report max 3 of these).
- Be selective: max ~12 findings, each genuinely worth acting on. Note tracked_in when an issue/PR/doc already tracks it (tracked is NOT done - still report). Your final message is data for an orchestrator, not prose for a human.

YOUR DIMENSION: full audit of sports-obsreport. Repo: C:/Users/avidu/Projects/khelsutra-guru/sports-obsreport.
Small private Python lib (observability/reporting; HEAD = v0.1.3 configurable item_noun). It is a git+ssh dependency of the engine (C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer) - the engine suite skips 12 tests when it's absent. Audit: how the engine pins it (grep pyproject/requirements in the engine for obsreport - pinned tag/commit or floating branch? floating = P1 reproducibility debt), release/tag hygiene (git tag -l vs versions), CI state, test coverage, API surface stability, README accuracy.

2. user (2026-07-02T02:47:18.708Z)

```
1f72c1c feat: configurable highlights item_noun (v0.1.3)
2f5a214 feat: spec-level customer_verdict override (v0.1.2)
cba67f2 feat: customer_message ? curated customer-facing flag text (v0.1.1)
7c40654 fix: correct setuptools build-backend (underscore) so editable/pip install works
d3e2a51 C4: RunRecorder + stable-diff + JSON Schema contract
911adf7 C3: spec-driven markdown renderers (technical + customer) + safe IO
918a728 C2: report config language ? spec loader + restricted rule engine
687b52e C1: scaffold obsreport ? typed envelope + deterministic JSON renderer
v0.1.0
v0.1.1
v0.1.2
v0.1.3
* main
  remotes/origin/main
```

3. user (2026-07-02T02:47:20.099Z)

```
total 29
drwxr-xr-x 1 avidu 197609    0 Jun  7 13:13 .
drwxr-xr-x 1 avidu 197609    0 Jun 29 20:32 ..
drwxr-xr-x 1 avidu 197609    0 Jul  2 08:17 .git
-rw-r--r-- 1 avidu 197609  169 Jun  7 12:37 .gitignore
drwxr-xr-x 1 avidu 197609    0 Jun  7 12:39 .pytest_cache
-rw-r--r-- 1 avidu 197609 1066 Jun  7 12:37 LICENSE
-rw-r--r-- 1 avidu 197609 1589 Jun  7 12:38 README.md
-rw-r--r-- 1 avidu 197609  764 Jun  7 13:17 pyproject.toml
drwxr-xr-x 1 avidu 197609    0 Jun  7 12:49 schema
drwxr-xr-x 1 avidu 197609    0 Jun  7 12:49 scripts
drwxr-xr-x 1 avidu 197609    0 Jun  7 12:47 specs
drwxr-xr-x 1 avidu 197609    0 Jun  7 12:37 src
drwxr-xr-x 1 avidu 197609    0 Jun  7 13:17 tests
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/.gitignore
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/.pytest_cache/.gitignore
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/.pytest_cache/CACHEDIR.TAG
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/.pytest_cache/README.md
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/.pytest_cache/v/cache/nodeids
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/LICENSE
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/pyproject.toml
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/README.md
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/schema/report_envelope.schema.json
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/scripts/gen_schema.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/specs/example.yaml
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/src/obsreport/diff.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/src/obsreport/envelope.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/src/obsreport/io.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/src/obsreport/recorder.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/src/obsreport/render/json_render.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/src/obsreport/render/markdown.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/src/obsreport/render/__init__.py
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/src/obsreport/render/__pycache__/json_render.cpython-313.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/src/obsreport/render/__pycache__/markdown.cpython-313.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/src/obsreport/render/__pycache__/__init__.cpython-313.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/src/obsreport/rules.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/src/obsreport/spec.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/src/obsreport/version.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/src/obsreport/__init__.py
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/src/obsreport/__pycache__/diff.cpython-313.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/src/obsreport/__pycache__/envelope.cpython-313.pyc
```

```
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/src/obsreport/__pycache__/io.cpython-313.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/src/obsreport/__pycache__/recorder.cpython-313.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/src/obsreport/__pycache__/rules.cpython-313.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/src/obsreport/__pycache__/spec.cpython-313.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/src/obsreport/__pycache__/version.cpython-313.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/src/obsreport/__pycache__/__init__.cpython-313.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/tests/test_customer_message.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/tests/test_diff.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/tests/test_envelope.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/tests/test_io.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/tests/test_recorder.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/tests/test_render_json.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/tests/test_render_markdown.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/tests/test_rules.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/tests/test_schema.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/tests/test_spec.py
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/tests/__pycache__/test_customer_message.cpython-313-pytest-9.0.3.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/tests/__pycache__/test_diff.cpython-313-pytest-9.0.3.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/tests/__pycache__/test_envelope.cpython-313-pytest-9.0.3.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/tests/__pycache__/test_io.cpython-313-pytest-9.0.3.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/tests/__pycache__/test_recorder.cpython-313-pytest-9.0.3.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/tests/__pycache__/test_render_json.cpython-313-pytest-9.0.3.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/tests/__pycache__/test_render_markdown.cpython-313-pytest-9.0.3.pyc
```

4. user (2026-07-02T02:47:30.190Z)

```
v0.1.3
1f72c1c2dd471d4e3ed3130f0a4040a6d4813042
821a664f5d2861397d5de7ebe803e72f71e9964a
.
..
.git
.gitignore
.pytest_cache
LICENSE
README.md
pyproject.toml
schema
scripts
specs
src
tests
```

5. user (2026-07-02T02:47:30.203Z)

```
1       [build-system]
2       requires = ["setuptools>=61.0.0"]
3       build-backend = "setuptools.build_meta"
4
5       [project]
6       name = "obsreport"
7       version = "0.1.3"
```

```
8       description = "Reusable per-run observability report toolkit: a small config-language
for emitting technical + customer-friendly reports across data/video pipelines."
9       readme = "README.md"
10      requires-python = ">=3.9"
11      license = { text = "MIT" }
12      authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
13      dependencies = [
14          "pydantic>=2.0.0",
15          "jinja2>=3.1.0",
16          "pyyaml>=6.0",
17      ]
18
19      [project.optional-dependencies]
20      dev = [
21          "pytest>=7.0.0",
22          "pytest-cov>=4.0.0",
23      ]
24
25      [tool.setuptools.packages.find]
26      where = ["src"]
27
28      [tool.pytest.ini_options]
29      pythonpath = ["src"]
30      testpaths = ["tests"]
31
```

6. user (2026-07-02T02:47:30.204Z)

```
1       # obsreport
2
3       A small, reusable toolkit for emitting **rich per-run observability reports** from
4       data/video pipelines ? designed so multiple tools produce reports in the **same
5       shape**, making them diffable across runs and across tools.
6
7       It is deliberately **not** a heavy data-quality framework (Great Expectations,
8       Evidently, Soda, ?). Those model tabular datasets; this models *a single pipeline
9       run as an ordered list of heterogeneous stages* plus a declarative
10      "footgun ? flag" rule layer. It is built only on permissive primitives
11      (`pydantic`, `Jinja2`, `PyYAML`) so it is safe for commercial use.
12
13      ## What you get
14
15      - **One typed envelope** (`ReportEnvelope`) that is the single source of truth.
16        Both the technical and the customer-friendly report render from the same object,
17        so they can never disagree on facts.
18      - **A declarative report config language** (YAML): describe report sections and
19        *rules* that turn known failure modes into flags, each citing a FAQ section so a
20        reader can self-debug. *(spec.py / rules.py ? added in C2.)*
21      - **Renderers**: deterministic JSON (machine source of truth, today) + Markdown
22        technical & customer views *(C3)*.
23      - **A RunRecorder** to capture stages in-flight, plus post-hoc harvest helpers,
24        with writes that **never raise** so reporting can't break the host run *(C4)*.
25
26      ## Status
27
28      Early scaffolding. Phase C1 (envelope + JSON renderer) is in place; C2?C4 follow.
29      See the consuming repos (`badminton-highlight-indexer`, `sports-data-collector`)
30      for the integration.
31
32      ## Develop
33
34      ```bash
35      pip install -e ".[dev]"
36      pytest
37      ```
```

38

7. user (2026-07-02T02:47:34.608Z)

```
Found 33 files
tests\test_api_endpoints.py
docs\CODE_MAP.md
backend\main.py
README.md
.github\workflows\ci.yml
tests\test_reporting_harvest.py
tests\test_reporting_spec_faq.py
tests\test_reporting.py
tests\test_run_signals.py
tests\test_person_detector.py
tests\test_async_jobs.py
deploy\digitalocean\bootstrap.sh
docs\CODE_CLEANUP_AUDIT_2026-06-24.md
pyproject.toml
requirements.txt
run_all_tests.py
backend\reporting\run_signals.py
backend\reporting\__init__.py
backend\reporting\harvest.py
docs\archives\decisions\DECISIONS.md
docs\ROADMAP.md
docs\PER_VIDEO_WORKSPACE.md
docs\PACKAGING_PLAN.md
docs\COMMERCIALIZATION.md
docs\BACKLOG.md
docs\POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md
docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md
docs\archives\DEFERRED_HARDENING_PLAN-COMPLETED-2026-06-14.md
docs\archives\HARDENING_LOOP_HANDOFF-COMPLETED-2026-06-14.md
backend\reporting\spec.yaml
docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md
docs\archives\quality-iterations\2026-06-initial\README.md
docs\archives\checkpoints\2026-06-logging-reporting\README.md
```

8. user (2026-07-02T02:47:45.373Z)

```
1f72c1c2dd471d4e3ed3130f0a4040a6d4813042
7a340d578f97ace3e1bd66bb9e63dacccecbb23f        refs/tags/v0.1.0
7c40654cf3cf233bb1daf7512066b8ce3cf1a5a8        refs/tags/v0.1.0^{}
2f98bc5fa9b34f632f8135ea20c00f596b3996ef        refs/tags/v0.1.1
cba67f29d261903e03026fe5fefa56ade63860a4        refs/tags/v0.1.1^{}
75aea1b807a79cf3669534808c3d48fb01753975        refs/tags/v0.1.2
2f5a214bf5051c386875a0e81bc724383b51d7e9        refs/tags/v0.1.2^{}
821a664f5d2861397d5de7ebe803e72f71e9964a        refs/tags/v0.1.3
1f72c1c2dd471d4e3ed3130f0a4040a6d4813042        refs/tags/v0.1.3^{}
origin  https://github.com/Khelsutra/sports-obsreport.git (fetch)
origin  https://github.com/Khelsutra/sports-obsreport.git (push)
1f72c1c feat: configurable highlights item_noun (v0.1.3)
2f5a214 feat: spec-level customer_verdict override (v0.1.2)
cba67f2 feat: customer_message ? curated customer-facing flag text (v0.1.1)
```

9. user (2026-07-02T02:47:45.499Z)

```
62-# Per-video observability reports (SOFT dependency: the pipeline degrades to a
63-# no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
64-# fetched over SSH; if you lack access, simply omit this group ? reporting
65-# tests skip via pytest.importorskip. For local dev against a sibling checkout:
66:#   pip install -e ../sports-obsreport --no-build-isolation
67-reporting = [
```

```
68:    "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
69-]
70-
71-# `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
72-# CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
--
89-# `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
90-indexer = "backend.main:main"
91-
92-[tool.hatch.metadata]
93:# The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
94-# reference*); hatchling requires opting in to allow those in metadata.
95-allow-direct-references = true
96-
97-[tool.hatch.build.targets.wheel]
```

10. user (2026-07-02T02:47:45.954Z)

```
31-
32-# Per-video observability reports (SOFT dependency: the pipeline degrades to a
33-# no-op if absent). Shared, permissively-licensed report toolkit. Pin a tag once
34-# the private repo is published, e.g.:
35:#   obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3
36-# For local development install it editable from a sibling checkout:
37:#   pip install -e ../sports-obsreport --no-build-isolation
```

11. user (2026-07-02T02:47:47.055Z)

```
131-        # NOTE: roboflow/trackers uses bare version tags (2.5.0), NOT v-prefixed.
132-        run: python -m pip install "trackers @
git+https://github.com/roboflow/trackers.git@2.5.0"
133-      - name: Install package (editable) + dev tooling
134-        # Editable PEP 621 install: runtime deps + the `dev` group
135-        # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
136-        run: python -m pip install -e .[dev]
137:      # obsreport (private, soft dep) is absent here: reporting tests skip via
138-      # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
139-      # is therefore calibrated against THIS lane's number (local runs with
140:      # obsreport installed measure a little higher).
141-      # Floor calibrated 2026-06-11 against this lane's measured 72% (local
142:      # with obsreport: 73%). Ratchet it UP as coverage grows; never down
143-      # without an owner decision.
144-      - name: Run full suite (with coverage floor)
145-        run: python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-
fail-under=70
146-
147-  # Cross-OS determinism guard for the committed golden regression fixtures
148-  # (docs/GOLDEN_REGRESSION_FIXTURES.md). The CONTROL replay path is pure-stdlib
```

12. user (2026-07-02T02:48:16.326Z)

```
{"defaultBranchRef":{"name":"main"},"isPrivate":true,"name":"sports-obsreport","owner":{"id":"O_
kgDOEYa5Fw","login":"Khelsutra"},"pushedAt":"2026-06-07T08:06:31Z","visibility":"PRIVATE"}
---
Khelsutra/sports-obsreport (redirect target)
---
0
---
---
```

13. user (2026-07-02T02:48:16.353Z)

```
1       """Version + report-schema version.
2
```

```
3       `SCHEMA_VERSION` is the contract version of the report ENVELOPE (bump the major
4       when a change is not backward compatible). `__version__` is the package release.
5       Consumers pin a git tag, so a schema bump can never surprise them silently.
6       """
7
8       SCHEMA_VERSION = "1.0"
9       __version__ = "0.1.3"
10
```

14. user (2026-07-02T02:48:16.374Z)

```
1       """obsreport ? reusable per-run observability reports.
2
3       Build a `ReportEnvelope` (typed, single source of truth), optionally run a
4       declarative rule spec to derive flags, then render it to JSON (machine) and
5       Markdown (technical + customer). Designed to be shared across pipelines so their
6       reports are diffable with each other and across runs.
7       """
8       from .version import SCHEMA_VERSION, __version__
9       from .envelope import (
10          Flag,
11          HighlightSummary,
12          Identity,
13          Metric,
14          ReportEnvelope,
15          Stage,
16          VideoMeta,
17          verdict_from_flags,
18      )
19      from .render.json_render import to_json
20      from .render.markdown import render_customer, render_technical
21      from .spec import Clause, ReportSpec, Rule, Section, load_spec, load_spec_dict,
load_spec_text
22      from .rules import apply_rules, run_spec
23      from .io import WriteResult, report_paths, write_report
24      from .recorder import RunRecorder
25      from .diff import stable_dict, stable_json
26
27      __all__ = [
28          "SCHEMA_VERSION",
29          "__version__",
30          "ReportEnvelope",
31          "Identity",
32          "VideoMeta",
33          "Stage",
34          "Metric",
35          "Flag",
36          "HighlightSummary",
37          "verdict_from_flags",
38          "to_json",
39          "ReportSpec",
40          "Section",
41          "Rule",
42          "Clause",
43          "load_spec",
44          "load_spec_text",
45          "load_spec_dict",
46          "apply_rules",
47          "run_spec",
48          "render_technical",
49          "render_customer",
50          "write_report",
51          "report_paths",
52          "WriteResult",
53          "RunRecorder",
```

```
        "stable_json",
        "stable_dict",
    ]

```

15. user (2026-07-02T02:48:29.137Z)

```
    """The rule engine: evaluate a spec's rules against an envelope to derive flags.

    Everything here is pure and dependency-free. A missing path makes a clause False
    for every operator except ``is_missing`` ? so a typo or an absent stage can never
    *accidentally* fire a footgun flag.
    """
    from __future__ import annotations

    from typing import Any, Dict, List

    from .envelope import Flag, ReportEnvelope
    from .spec import Clause, Rule, ReportSpec


    class _Missing:
        """Sentinel for an unresolved path (distinct from a real ``None`` value)."""

        _singleton = None

        def __repr__(self) -> str:  # pragma: no cover - cosmetic
            return "<MISSING>"


    MISSING = _Missing()


    def to_rule_context(env: ReportEnvelope) -> Dict[str, Any]:
        """Project an envelope into an ergonomic dict for dotted-path lookups.

        Lists that you'd want to address by name are re-keyed: ``stages`` by stage
        name, and each stage's ``metrics`` by metric key. So a rule can say
        ``stages.segmentation.metrics.segment_count`` or
        ``stages.ai_handoff.details.api_key_present``.
        """
        stages: Dict[str, Any] = {}
        for s in env.stages:
            stages[s.name] = {
                "status": s.status,
                "duration_s": s.duration_s,
                "messages": list(s.messages),
                "details": dict(s.details),
                "metrics": {m.key: m.value for m in s.metrics},
            }
        return {
            "identity": env.identity.model_dump(mode="json"),
            "video_meta": env.video_meta.model_dump(mode="json"),
            "highlights": env.highlights.model_dump(mode="json") if env.highlights else {},
            "verdict": env.verdict,
            "extras": dict(env.extras),
            "stages": stages,
            "flags": [f.model_dump(mode="json") for f in env.flags],
        }


    def get_path(data: Any, dotted: str) -> Any:
        """Navigate ``data`` by a dotted path; return MISSING if any hop is absent.
        Numeric path parts index into lists."""
        cur = data
```

```
        for part in dotted.split("."):
            if isinstance(cur, dict):
                if part in cur:
                    cur = cur[part]
                else:
                    return MISSING
            elif isinstance(cur, list):
                try:
                    idx = int(part)
                except ValueError:
                    return MISSING
                if -len(cur) <= idx < len(cur):
                    cur = cur[idx]
                else:
                    return MISSING
            else:
                return MISSING
        return cur


    def _ordered(op: str, a: Any, b: Any) -> bool:
        try:
            if op == "gt":
                return a > b
            if op == "ge":
                return a >= b
            if op == "lt":
                return a < b
            if op == "le":
                return a <= b
        except TypeError:
            return False
        return False


    def eval_clause(ctx: Dict[str, Any], clause: Clause) -> bool:
        actual = get_path(ctx, clause.path)
        op = clause.op

        if op == "is_missing":
            return actual is MISSING
        if op == "is_present":
            return actual is not MISSING
        if actual is MISSING:
            # Every other operator is False against a missing path (no accidental fires).
            return False

        if op == "eq":
            return actual == clause.value
        if op == "ne":
            return actual != clause.value
        if op in ("gt", "ge", "lt", "le"):
            return _ordered(op, actual, clause.value)
        if op == "in":
            try:
                return actual in clause.value
            except TypeError:
                return False
        if op == "contains":
            try:
                return clause.value in actual
            except TypeError:
                return False
        if op in ("eq_path", "ne_path"):
            other = get_path(ctx, clause.value) if isinstance(clause.value, str) else
```

```
MISSING
            if other is MISSING:
                return False
            return actual == other if op == "eq_path" else actual != other
        return False


    def eval_rule(ctx: Dict[str, Any], rule: Rule) -> bool:
        """A rule fires when ALL its clauses hold (logical AND)."""
        return all(eval_clause(ctx, c) for c in rule.when)


    def _evidence(ctx: Dict[str, Any], rule: Rule) -> Dict[str, Any]:
        """Snapshot the values that triggered the rule, so the flag is self-explaining
        in the report (MISSING rendered as None for JSON-safety)."""
        ev: Dict[str, Any] = {}
        for c in rule.when:
            val = get_path(ctx, c.path)
            ev[c.path] = None if val is MISSING else val
        return ev


    def apply_rules(env: ReportEnvelope, spec: ReportSpec) -> List[Flag]:
        """Return the flags a spec's rules produce for an envelope (no mutation)."""
        ctx = to_rule_context(env)
        flags: List[Flag] = []
        for rule in spec.rules:
            if eval_rule(ctx, rule):
                flags.append(
                    Flag(
                        code=rule.code,
                        severity=rule.severity,
                        message=rule.message,
                        stage=rule.stage,
                        faq_ref=rule.faq_ref,
                        customer_message=rule.customer_message,
                        evidence=_evidence(ctx, rule),
                    )
                )
        return flags


    def run_spec(env: ReportEnvelope, spec: ReportSpec) -> ReportEnvelope:
        """Apply a spec's rules to ``env`` in place, then recompute the verdict."""
        env.flags.extend(apply_rules(env, spec))
        env.recompute_verdict()
        return env

```

16. user (2026-07-02T02:48:29.614Z)

```
    """File output: the path convention + safe writes.

    Convention (next to the video by default):
        /videos/match.mp4  ->  /videos/match.report.json     (machine, source of truth)
                               /videos/match.report.md        (technical, debuggable)
                               /videos/match.summary.md       (customer-friendly)

    Overridable with ``output_dir``. For remote/no-local-file runs (e.g. the
    collector before download), output falls back to ``output_dir`` or
    ``reports/<video_id>/``.

    THE CARDINAL RULE: writing a report must NEVER break the host pipeline run.
    Every write is wrapped; failures are collected into a ``WriteResult`` and
    returned, never raised.
```

```
    """
    from __future__ import annotations

    import os
    from dataclasses import dataclass, field
    from typing import Dict, List, Optional

    from .envelope import ReportEnvelope
    from .render.json_render import to_json
    from .render.markdown import render_customer, render_technical
    from .spec import ReportSpec


    @dataclass
    class WriteResult:
        ok: bool
        paths: Dict[str, str] = field(default_factory=dict)
        errors: List[str] = field(default_factory=list)


    def _base(video_path: Optional[str], output_dir: Optional[str], video_id: Optional[str],
stem: Optional[str]):
        """Return (directory, basename) for the report files."""
        if stem:
            name = stem
        elif video_path:
            name = os.path.splitext(os.path.basename(video_path))[0]
        elif video_id:
            name = video_id
        else:
            name = "report"

        if output_dir:
            return output_dir, name
        if video_path:
            return os.path.dirname(os.path.abspath(video_path)) or ".", name
        # remote / no local file
        return os.path.join("reports", video_id or "run"), name


    def report_paths(
        video_path: Optional[str] = None,
        output_dir: Optional[str] = None,
        video_id: Optional[str] = None,
        stem: Optional[str] = None,
    ) -> Dict[str, str]:
        """Resolve the three report file paths (no IO)."""
        directory, name = _base(video_path, output_dir, video_id, stem)
        return {
            "json": os.path.join(directory, f"{name}.report.json"),
            "technical_md": os.path.join(directory, f"{name}.report.md"),
            "customer_md": os.path.join(directory, f"{name}.summary.md"),
        }


    def _write_text(path: str, text: str) -> None:
        """Atomic write: temp file in the same dir, then os.replace. Patchable in tests."""
        directory = os.path.dirname(path) or "."
        os.makedirs(directory, exist_ok=True)
        tmp = f"{path}.tmp"
        with open(tmp, "w", encoding="utf-8", newline="\n") as f:
            f.write(text)
        os.replace(tmp, path)


```

```
    def write_report(
        envelope: ReportEnvelope,
        spec: Optional[ReportSpec] = None,
        *,
        video_path: Optional[str] = None,
        output_dir: Optional[str] = None,
        video_id: Optional[str] = None,
        stem: Optional[str] = None,
        write_customer: bool = True,
    ) -> WriteResult:
        """Render + write json / technical md / (optional) customer md. Never raises."""
        result = WriteResult(ok=True)
        try:
            paths = report_paths(
                video_path=video_path,
                output_dir=output_dir,
                video_id=video_id or envelope.identity.video_id,
                stem=stem,
            )
        except Exception as e:  # pragma: no cover - defensive
            return WriteResult(ok=False, errors=[f"path resolution failed: {e}"])

        jobs = [("json", paths["json"], lambda: to_json(envelope)),
                ("technical_md", paths["technical_md"], lambda: render_technical(envelope,
spec))]
        if write_customer:
            jobs.append(("customer_md", paths["customer_md"], lambda:
render_customer(envelope, spec)))

        for key, path, render in jobs:
            try:
                _write_text(path, render())
                result.paths[key] = path
            except Exception as e:
                result.ok = False
                result.errors.append(f"{key}: {e}")
        return result

```

17. user (2026-07-02T02:48:37.451Z)

```
    """Typed report envelope ? the single source of truth for one observability report.

    A report describes ONE processed video / one pipeline run as an ordered list of
    heterogeneous *stages*, plus video metadata, derived *flags* (warnings/errors),
    a customer-safe *highlight summary*, and an overall *verdict*.

    Both the technical and the customer-friendly renderers consume this SAME object,
    so the two reports can never disagree on facts. This module is pure data + a few
    pure helpers ? there is NO file/network IO here (see `io.py`) and nothing runs at
    import time.
    """
    from __future__ import annotations

    from typing import Any, Dict, List, Literal, Optional

    from pydantic import BaseModel, Field

    from .version import SCHEMA_VERSION

    # --- closed vocabularies (kept as Literals so typos fail fast in consumers) ---
    Audience = Literal["technical", "customer", "both"]
    StageStatus = Literal["ok", "warn", "error", "skipped", "not_run"]
    Severity = Literal["error", "warn", "info"]
    Verdict = Literal["pass", "pass_with_warnings", "fail"]
```

```
    FpsSource = Literal["probed", "fallback", "declared", "unknown"]
    VideoIdKind = Literal["file_md5", "human_id", "unknown"]


    class Metric(BaseModel):
        """A single named measurement attached to a stage.

        `audience` controls which report it surfaces in: technical-only by default;
        use ``"both"`` for things a customer should also see (e.g. duration).
        """

        key: str
        value: Any = None
        unit: Optional[str] = None
        audience: Audience = "technical"


    class Stage(BaseModel):
        """One step the video went through (validate, segment, stitch, acquire, ...)."""

        name: str
        status: StageStatus = "ok"
        started_at: Optional[str] = None
        duration_s: Optional[float] = None
        metrics: List[Metric] = Field(default_factory=list)
        details: Dict[str, Any] = Field(default_factory=dict)
        messages: List[str] = Field(default_factory=list)

        def add_metric(
            self,
            key: str,
            value: Any,
            unit: Optional[str] = None,
            audience: Audience = "technical",
        ) -> "Stage":
            """Append a metric and return self (chainable)."""
            self.metrics.append(Metric(key=key, value=value, unit=unit, audience=audience))
            return self


    class Flag(BaseModel):
        """A derived warning/error ? the debugging intelligence of the report.

        `faq_ref` points at the README/FAQ section a reader should open to self-debug
        (e.g. ``"README#Q7"``). It is REQUIRED for error/warn severities; this is
        enforced by `ReportEnvelope.lint`, not by the constructor, so harvest code can
        build a flag incrementally before deciding its citation.
        """

        code: str
        severity: Severity
        message: str
        stage: Optional[str] = None
        faq_ref: Optional[str] = None
        evidence: Dict[str, Any] = Field(default_factory=dict)
        # Plain-language phrasing for the customer report. The customer view shows ONLY
        # flags that set this (so internal/technical findings never leak); the technical
        # view always uses `message`.
        customer_message: Optional[str] = None


    class Identity(BaseModel):
        """Who/what this report is about. `video_id_kind` records the id SEMANTICS so a
        cross-tool diff (indexer file-MD5 vs collector human id) never mis-joins."""

```

```
        tool: str
        schema_version: str = SCHEMA_VERSION
        tool_version: Optional[str] = None
        video_id: Optional[str] = None
        video_id_kind: VideoIdKind = "unknown"
        video_path: Optional[str] = None
        video_url: Optional[str] = None
        generated_at: Optional[str] = None
        run_id: Optional[str] = None


    class VideoMeta(BaseModel):
        """Probed/declared media facts. `fps_source` distinguishes a real probe from a
        silent fallback (a known footgun the report exists to surface)."""

        duration_s: Optional[float] = None
        fps: Optional[float] = None
        fps_source: FpsSource = "unknown"
        resolution: Optional[str] = None
        width: Optional[int] = None
        height: Optional[int] = None
        container: Optional[str] = None
        size_mb: Optional[float] = None


    class HighlightSummary(BaseModel):
        """Customer-safe, meta-level summary of what the tool produced (no internals)."""

        segment_count: int = 0
        total_highlight_s: float = 0.0
        coverage_pct: Optional[float] = None
        notes: List[str] = Field(default_factory=list)


    def verdict_from_flags(flags: List[Flag]) -> Verdict:
        """Pure verdict derivation: any error -> fail; any warn -> pass_with_warnings."""
        severities = {f.severity for f in flags}
        if "error" in severities:
            return "fail"
        if "warn" in severities:
            return "pass_with_warnings"
        return "pass"


    class ReportEnvelope(BaseModel):
        """The whole report. Serialize with `obsreport.to_json` (machine source of
        truth) or render via the markdown renderers (human/customer views)."""

        identity: Identity
        video_meta: VideoMeta = Field(default_factory=VideoMeta)
        stages: List[Stage] = Field(default_factory=list)
        flags: List[Flag] = Field(default_factory=list)
        highlights: Optional[HighlightSummary] = None
        verdict: Verdict = "pass"
        extras: Dict[str, Any] = Field(default_factory=dict)

        # --- pure helpers (no IO) ---
        def get_stage(self, name: str) -> Optional[Stage]:
            for s in self.stages:
                if s.name == name:
                    return s
            return None

        def recompute_verdict(self) -> "ReportEnvelope":
            """Set `verdict` from current flags. Call in finalize, after rules run."""
```

```
            self.verdict = verdict_from_flags(self.flags)
            return self

        def lint(self) -> List[str]:
            """Structural self-checks. Empty list == clean. Used by tests and the
            recorder's finalize to guarantee a well-formed, debuggable report."""
            issues: List[str] = []
            for f in self.flags:
                if f.severity in ("error", "warn") and not f.faq_ref:
                    issues.append(
                        f"flag '{f.code}' has severity '{f.severity}' but no faq_ref "
                        f"(error/warn flags must cite a README/FAQ section)"
                    )
            expected = verdict_from_flags(self.flags)
            if self.verdict != expected:
                issues.append(
                    f"verdict '{self.verdict}' does not match flag-derived '{expected}' "
                    f"(did you forget recompute_verdict()?)"
                )
            return issues

```

18. user (2026-07-02T02:48:38.074Z)

```
"""RunRecorder ? capture an envelope incrementally during a pipeline run.

A tool creates a recorder at the top of a run, opens a ``stage(...)`` per pipeline
step (auto-timed; exceptions are captured into the stage, NOT re-raised, so
reporting can't crash the run), fills in video metadata / highlights / manual
flags, then ``finalize()`` (runs the spec's rules + computes the verdict) and
``write()`` (emits the files). The clock is injectable so tests are deterministic.

Tool-specific *post-hoc harvest* (rebuilding an envelope from a DB / artifacts)
lives in each consuming repo ? it just drives this same recorder API.
"""
from __future__ import annotations

import time
from contextlib import contextmanager
from datetime import datetime, timezone
from typing import Any, Callable, Dict, Iterator, Optional

from .envelope import Flag, HighlightSummary, Identity, ReportEnvelope, Stage
from .io import WriteResult, write_report
from .rules import run_spec
from .spec import ReportSpec


def _utc_now_iso() -> str:
    return datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")


class RunRecorder:
    def __init__(
        self,
        tool: str,
        *,
        tool_version: Optional[str] = None,
        video_id: Optional[str] = None,
        video_id_kind: str = "unknown",
        video_path: Optional[str] = None,
        video_url: Optional[str] = None,
        run_id: Optional[str] = None,
        spec: Optional[ReportSpec] = None,
        now: Callable[[], str] = _utc_now_iso,
```

```
            monotonic: Callable[[], float] = time.perf_counter,
        ) -> None:
            self._now = now
            self._monotonic = monotonic
            self.spec = spec
            self._finalized = False
            self.envelope = ReportEnvelope(
                identity=Identity(
                    tool=tool,
                    tool_version=tool_version,
                    video_id=video_id,
                    video_id_kind=video_id_kind,  # type: ignore[arg-type]
                    video_path=video_path,
                    video_url=video_url,
                    run_id=run_id,
                    generated_at=now(),
                )
            )

        # --- media / highlights ---
        def set_video_meta(self, **kw: Any) -> "RunRecorder":
            self.envelope.video_meta = self.envelope.video_meta.model_copy(update=kw)
            return self

        def set_highlights(self, **kw: Any) -> "RunRecorder":
            cur = self.envelope.highlights or HighlightSummary()
            self.envelope.highlights = cur.model_copy(update=kw)
            return self

        # --- stages ---
        def get_or_create_stage(self, name: str) -> Stage:
            st = self.envelope.get_stage(name)
            if st is None:
                st = Stage(name=name)
                self.envelope.stages.append(st)
            return st

        @contextmanager
        def stage(self, name: str, *, capture_exceptions: bool = True) -> Iterator[Stage]:
            """Time a pipeline step. On exception, mark the stage 'error' and record the
            message; re-raise only if ``capture_exceptions=False``."""
            st = self.get_or_create_stage(name)
            st.started_at = self._now()
            t0 = self._monotonic()
            try:
                yield st
            except Exception as e:  # noqa: BLE001 - we deliberately swallow to protect the
run
                st.status = "error"
                st.messages.append(f"{type(e).__name__}: {e}")
                if not capture_exceptions:
                    raise
            finally:
                st.duration_s = round(self._monotonic() - t0, 6)

        # --- flags ---
        def add_flag(
            self,
            code: str,
            severity: str,
            message: str,
            *,
            faq_ref: Optional[str] = None,
            stage: Optional[str] = None,
            evidence: Optional[Dict[str, Any]] = None,
```

```
            customer_message: Optional[str] = None,
        ) -> "RunRecorder":
            self.envelope.flags.append(
                Flag(
                    code=code,
                    severity=severity,  # type: ignore[arg-type]
                    message=message,
                    faq_ref=faq_ref,
                    stage=stage,
                    evidence=evidence or {},
                    customer_message=customer_message,
                )
            )
            return self

        # --- finalize / write ---
        def finalize(self) -> ReportEnvelope:
            """Apply the spec's rules (if any) and compute the verdict. Idempotent."""
            if not self._finalized:
                if self.spec is not None:
                    run_spec(self.envelope, self.spec)
                else:
                    self.envelope.recompute_verdict()
                self._finalized = True
            return self.envelope

        def write(self, **io_kwargs: Any) -> WriteResult:
            """Finalize then write all report files. Never raises (see io.write_report)."""
            self.finalize()
            kwargs: Dict[str, Any] = dict(
                video_path=self.envelope.identity.video_path,
                video_id=self.envelope.identity.video_id,
            )
            kwargs.update(io_kwargs)
            return write_report(self.envelope, self.spec, **kwargs)

```

19. user (2026-07-02T02:48:47.933Z)

```
    """The report config language: a YAML spec describing report *sections* (layout)
    and *rules* (footgun -> flag).

    A spec is pure data validated into pydantic models. Specs are authored by us (one
    per tool, under ``specs/``), so we validate strictly and fail fast: every
    actionable rule (error/warn) MUST cite a FAQ section, and every rule MUST have at
    least one condition. That guarantee is what makes the resulting reports
    self-debuggable.
    """
    from __future__ import annotations

    from pathlib import Path
    from typing import Any, Dict, List, Literal, Optional, Union

    import yaml
    from pydantic import BaseModel, Field, model_validator

    from .envelope import Audience, Severity

    # Comparison operators the rule engine understands. Deliberately a closed,
    # sub-Turing set ? there is NO expression grammar and NO eval anywhere.
    ClauseOp = Literal[
        "eq", "ne", "gt", "ge", "lt", "le",
        "in", "contains",
        "is_missing", "is_present",
        "eq_path", "ne_path",
```

```
    ]


    class Clause(BaseModel):
        """One condition: compare the value at ``path`` against ``value``.

        ``path`` is a dotted accessor into the rule context (see ``rules.to_rule_context``),
        e.g. ``stages.ai_handoff.metrics.segment_count``. For ``eq_path``/``ne_path``,
        ``value`` is itself a dotted path (compare two fields)."""

        path: str
        op: ClauseOp
        value: Any = None


    class Rule(BaseModel):
        """A footgun detector. If every clause in ``when`` holds, emit a flag."""

        code: str
        severity: Severity
        message: str
        when: List[Clause] = Field(default_factory=list)
        faq_ref: Optional[str] = None
        stage: Optional[str] = None
        customer_message: Optional[str] = None  # if set, this finding also shows (gently)
to customers

        @model_validator(mode="after")
        def _validate(self) -> "Rule":
            if self.severity in ("error", "warn") and not self.faq_ref:
                raise ValueError(
                    f"rule '{self.code}' has severity '{self.severity}' but no faq_ref "
                    f"(actionable flags must cite a README/FAQ section so a reader can self-
debug)"
                )
            if not self.when:
                raise ValueError(
                    f"rule '{self.code}' has no conditions; a rule must have >=1 clause "
                    f"(use is_present/is_missing for existence checks)"
                )
            return self


    SectionKind = Literal[
        "identity", "video_meta", "highlights", "verdict", "flags", "stages", "stage"
    ]


    class Section(BaseModel):
        """A block in the rendered report. The renderer walks these in order."""

        id: str
        title: str
        audience: Audience = "technical"
        kind: SectionKind = "stages"
        stage: Optional[str] = None  # required when kind == "stage"
        metrics: Optional[List[str]] = None  # optional whitelist of metric keys to show
        item_noun: str = "highlight"  # noun for a HighlightSummary item (e.g. "rally",
"point")

        @model_validator(mode="after")
        def _validate(self) -> "Section":
            if self.kind == "stage" and not self.stage:
                raise ValueError(f"section '{self.id}' is kind=stage but names no 'stage'")
            return self
```

```
89
90
91      class ReportSpec(BaseModel):
92          name: str
93          title: Optional[str] = None
94          sections: List[Section] = Field(default_factory=list)
95          rules: List[Rule] = Field(default_factory=list)
96          # Optional per-tool phrasing of the customer-facing verdict, keyed by verdict
97          # (pass / pass_with_warnings / fail). Lets each tool speak to its own audience
98          # (e.g. a highlight viewer vs an annotation vendor). Falls back to a generic
default.
99          customer_verdict: Optional[Dict[str, str]] = None
100
101
102     def load_spec_dict(data: dict) -> ReportSpec:
103         return ReportSpec.model_validate(data or {})
104
105
106     def load_spec_text(text: str) -> ReportSpec:
107         return load_spec_dict(yaml.safe_load(text) or {})
108
109
110     def load_spec(path: Union[str, Path]) -> ReportSpec:
111         return load_spec_text(Path(path).read_text(encoding="utf-8"))
112
```

20. user (2026-07-02T02:48:48.827Z)

```
1       """Regenerate the checked-in JSON Schema for the report envelope.
2
3       Run after changing envelope.py:  python scripts/gen_schema.py
4       The schema is the published contract; test_schema.py fails if it drifts.
5       """
6       import json
7       import sys
8       from pathlib import Path
9
10      ROOT = Path(__file__).resolve().parent.parent
11      sys.path.insert(0, str(ROOT / "src"))
12
13      from obsreport.envelope import ReportEnvelope  # noqa: E402
14
15      OUT = ROOT / "schema" / "report_envelope.schema.json"
16
17
18      def schema_text() -> str:
19          return json.dumps(ReportEnvelope.model_json_schema(), indent=2, sort_keys=True) +
"\n"
20
21
22      def main() -> None:
23          OUT.parent.mkdir(parents=True, exist_ok=True)
24          OUT.write_text(schema_text(), encoding="utf-8")
25          print(f"wrote {OUT}")
26
27
28      if __name__ == "__main__":
29          main()
30
```

21. user (2026-07-02T02:48:49.581Z)

```
Found 13 files
C:\Users\avidu\Projects\khelsutra-guru\sports-data-collector\tests\test_reporting_noraise.py
C:\Users\avidu\Projects\khelsutra-guru\sports-data-collector\src\reporting\__init__.py
```

```
C:\Users\avidu\Projects\khelsutra-guru\sports-data-collector\src\reporting\adapter.py
C:\Users\avidu\Projects\khelsutra-guru\sports-data-collector\pyproject.toml
C:\Users\avidu\Projects\khelsutra-guru\sports-data-collector\sports-data-
collector\tests\test_reporting_faq.py
C:\Users\avidu\Projects\khelsutra-guru\sports-data-collector\sports-data-
collector\tests\test_reporting.py
C:\Users\avidu\Projects\khelsutra-guru\sports-data-collector\sports-data-
collector\src\reporting\adapter.py
C:\Users\avidu\Projects\khelsutra-guru\sports-data-collector\sports-data-
collector\src\reporting\spec.yaml
C:\Users\avidu\Projects\khelsutra-guru\sports-data-collector\sports-data-
collector\src\reporting\__init__.py
C:\Users\avidu\Projects\khelsutra-guru\sports-data-collector\sports-data-
collector\pyproject.toml
C:\Users\avidu\Projects\khelsutra-guru\sports-data-collector\tests\test_reporting_faq.py
C:\Users\avidu\Projects\khelsutra-guru\sports-data-collector\tests\test_reporting.py
C:\Users\avidu\Projects\khelsutra-guru\sports-data-collector\src\reporting\spec.yaml
```

22. user (2026-07-02T02:49:01.281Z)

```
31-]
32-# Per-video observability reports (SOFT dependency: ingestion degrades to a no-op
33-# if absent). Shared, permissively-licensed report toolkit. Pin a tag once the
34-# private repo is published, e.g.:
35:#   obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3
36-# For local development install it editable from a sibling checkout:
37:#   pip install -e ../sports-obsreport --no-build-isolation
38-report = [
39:    "obsreport>=0.1.3",
40-]
```

23. user (2026-07-02T02:49:04.587Z)

```
schema in sync: True
version: 0.1.3
```

24. user (2026-07-02T02:49:04.607Z)

```
1       """Stable serialization for cross-run / cross-tool diffing.
2
3       The plain JSON is the machine source of truth, but it carries volatile fields
4       (timestamps, run id, per-stage durations) that change every run and would swamp a
5       diff. ``stable_json`` strips those so two runs of the same logical video diff only
6       where the *substance* differs ? which is the whole point of "evaluate output
7       differences."
8       """
9       from __future__ import annotations
10
11      import json
12      from typing import Any, Iterable, Set
13
14      from .envelope import ReportEnvelope
15
16      VOLATILE_KEYS: Set[str] = {"generated_at", "run_id", "started_at", "duration_s"}
17
18
19      def _strip(obj: Any, keys: Set[str]) -> Any:
20          if isinstance(obj, dict):
21              return {k: _strip(v, keys) for k, v in obj.items() if k not in keys}
22          if isinstance(obj, list):
23              return [_strip(v, keys) for v in obj]
24          return obj
25
26
```

```
27      def stable_dict(envelope: ReportEnvelope, extra_volatile: Iterable[str] = ()) -> dict:
28          keys = VOLATILE_KEYS | set(extra_volatile)
29          return _strip(envelope.model_dump(mode="json"), keys)
30
31
32      def stable_json(envelope: ReportEnvelope, indent: int = 2, extra_volatile: Iterable[str]
= ()) -> str:
33          return json.dumps(stable_dict(envelope, extra_volatile), indent=indent,
sort_keys=True, ensure_ascii=False)
34
```

25. user (2026-07-02T02:49:17.315Z)

```
404
{"message": "Not Found"}
```

26. user (2026-07-02T02:49:17.339Z)

```
1       """Markdown renderers ? technical and customer views of the SAME envelope.
2
3       Section selection/order/titles come from the report spec (or sensible defaults).
4       The technical view shows everything (stages, all flags with code+evidence+faq_ref);
5       the customer view shows only video metadata, meta-level highlights, and the
6       human-readable text of actionable flags ? never internal codes, evidence, or raw
7       stage dumps. Rendering is pure and deterministic (good for snapshot diffs).
8       """
9       from __future__ import annotations
10
11      import os
12      from typing import List, Optional
13
14      from ..envelope import Flag, ReportEnvelope, Stage
15      from ..spec import ReportSpec, Section
16
17      # Default layout when a spec ships no sections. Covers both audiences; the
18      # renderer filters by the requested audience.
19      DEFAULT_SECTIONS: List[Section] = [
20          Section(id="run", title="Run", kind="identity", audience="both"),
21          Section(id="video", title="Video", kind="video_meta", audience="both"),
22          Section(id="highlights", title="Highlights", kind="highlights",
audience="customer"),
23          Section(id="stages", title="Processing stages", kind="stages",
audience="technical"),
24          Section(id="findings", title="Findings", kind="flags", audience="technical"),
25          Section(id="issues", title="Things to know", kind="flags", audience="customer"),
26          Section(id="verdict", title="Verdict", kind="verdict", audience="both"),
27      ]
28
29      _VERDICT_LABEL = {
30          "pass": "PASS",
31          "pass_with_warnings": "PASS (with warnings)",
32          "fail": "FAIL",
33      }
34      # A customer should never see a bare "FAIL" ? for them the verdict is about whether
35      # their highlights are usable, framed plainly, with no internal blame.
36      _VERDICT_CUSTOMER = {
37          "pass": "? All good ? your highlights are ready.",
38          "pass_with_warnings": "? Your highlights are ready. (A couple of minor notes
above.)",
39          "fail": "?? We ran into a problem processing this video, so some highlights may be "
40                  "missing. See the notes above, or contact support.",
41      }
42      _SEVERITY_MARK = {"error": "[ERROR]", "warn": "[WARN]", "info": "[info]"}
43
44
```

```
    # --- small formatters -----------------------------------------------------
    def _num(v) -> str:
        if isinstance(v, float):
            return f"{v:.2f}".rstrip("0").rstrip(".")
        return str(v)


    def _secs(v) -> str:
        try:
            f = float(v)
        except (TypeError, ValueError):
            return "?"
        m, s = divmod(int(round(f)), 60)
        return f"{m}m{s:02d}s" if m else f"{f:.1f}s"


    def _kv_table(rows: List[tuple]) -> str:
        rows = [(k, v) for k, v in rows if v is not None and v != ""]
        if not rows:
            return "_(none)_"
        out = ["| Field | Value |", "| --- | --- |"]
        out += [f"| {k} | {v} |" for k, v in rows]
        return "\n".join(out)


    # --- per-kind builders ----------------------------------------------------
    def _identity(env: ReportEnvelope, audience: str) -> str:
        i = env.identity
        if audience == "customer":
            name = os.path.basename(i.video_path) if i.video_path else (i.video_url or
i.video_id or "?")
            return _kv_table([("Video", name), ("Report generated", i.generated_at)])
        return _kv_table([
            ("Tool", f"{i.tool} {i.tool_version or ''}".strip()),
            ("Video id", f"{i.video_id} ({i.video_id_kind})" if i.video_id else None),
            ("Video path", i.video_path),
            ("Video url", i.video_url),
            ("Run id", i.run_id),
            ("Generated", i.generated_at),
            ("Schema", i.schema_version),
        ])


    def _video_meta(env: ReportEnvelope, audience: str) -> str:
        m = env.video_meta
        if audience == "customer":
            return _kv_table([
                ("Duration", _secs(m.duration_s) if m.duration_s is not None else None),
                ("Resolution", m.resolution or (f"{m.width}x{m.height}" if m.width and
m.height else None)),
                ("Frame rate", f"{_num(m.fps)} fps" if m.fps else None),
            ])
        fps = f"{_num(m.fps)} fps ({m.fps_source})" if m.fps is not None else f"?
({m.fps_source})"
        return _kv_table([
            ("Duration", _secs(m.duration_s) if m.duration_s is not None else None),
            ("Resolution", m.resolution or (f"{m.width}x{m.height}" if m.width and m.height
else None)),
            ("FPS", fps),
            ("Container", m.container),
            ("Size", f"{_num(m.size_mb)} MB" if m.size_mb is not None else None),
        ])


    def _plural(noun: str, n: int) -> str:
```

```
        if n == 1:
            return noun
        return noun[:-1] + "ies" if noun.endswith("y") else noun + "s"


    def _highlights(env: ReportEnvelope, audience: str, section: Optional[Section] = None)
-> str:
        h = env.highlights
        noun = section.item_noun if section else "highlight"
        if not h:
            return f"_(no {_plural(noun, 0)} produced)_"
        lines = [
            f"- **{h.segment_count}** {_plural(noun, h.segment_count)}",
            f"- **{_secs(h.total_highlight_s)}** of play",
        ]
        if h.coverage_pct is not None:
            lines.append(f"- **{_num(h.coverage_pct)}%** of the video is {_plural(noun,
2)}")
        lines += [f"- {n}" for n in h.notes]
        return "\n".join(lines)


    def _stage_block(s: Stage, audience: str) -> str:
        head = f"**{s.name}** ? `{s.status}`"
        if s.duration_s is not None:
            head += f" ({_secs(s.duration_s)})"
        lines = [head]
        metrics = [m for m in s.metrics if audience != "customer" or m.audience in
("customer", "both")]
        for m in metrics:
            unit = f" {m.unit}" if m.unit else ""
            lines.append(f"  - {m.key}: {_num(m.value)}{unit}")
        if audience != "customer":
            for k, v in s.details.items():
                lines.append(f"  - _{k}_: {v}")
        for msg in s.messages:
            lines.append(f"  - {msg}")
        return "\n".join(lines)


    def _stages(env: ReportEnvelope, audience: str, only: Optional[str] = None) -> str:
        stages = env.stages if only is None else [s for s in env.stages if s.name == only]
        if not stages:
            return "_(no stages recorded)_"
        return "\n\n".join(_stage_block(s, audience) for s in stages)


    def _flags(env: ReportEnvelope, audience: str) -> str:
        if audience == "customer":
            # Only show findings explicitly curated for customers (those with a
            # customer_message). Internal/technical flags never leak into this view.
            actionable = [f for f in env.flags if f.severity in ("error", "warn") and
f.customer_message]
            if not actionable:
                return "_Nothing needs your attention._"
            mark = {"error": "?", "warn": "??"}
            return "\n".join(f"- {mark[f.severity]} {f.customer_message}" for f in
actionable)
        if not env.flags:
            return "_No findings._"
        blocks = []
        for f in env.flags:
            line = f"- {_SEVERITY_MARK[f.severity]} **{f.code}** ? {f.message}"
            if f.faq_ref:
                line += f"  \n  ? see **{f.faq_ref}**"
```

```
            if f.evidence:
                ev = ", ".join(f"{k}={v}" for k, v in f.evidence.items())
                line += f"  \n  ? evidence: {ev}"
            blocks.append(line)
        return "\n".join(blocks)


    def _verdict(env: ReportEnvelope, audience: str, spec: Optional[ReportSpec] = None) ->
str:
        if audience == "customer":
            if spec and spec.customer_verdict and env.verdict in spec.customer_verdict:
                return spec.customer_verdict[env.verdict]
            return _VERDICT_CUSTOMER.get(env.verdict, env.verdict)
        return f"**{_VERDICT_LABEL.get(env.verdict, env.verdict)}**"


    _BUILDERS = {
        "identity": _identity,
        "video_meta": _video_meta,
        "highlights": _highlights,
        "flags": _flags,
        "verdict": _verdict,
    }


    def _build_section(env: ReportEnvelope, section: Section, audience: str,
                       spec: Optional[ReportSpec] = None) -> str:
        if section.kind == "stages":
            return _stages(env, audience)
        if section.kind == "stage":
            return _stages(env, audience, only=section.stage)
        if section.kind == "verdict":
            return _verdict(env, audience, spec)
        if section.kind == "highlights":
            return _highlights(env, audience, section)
        return _BUILDERS[section.kind](env, audience)


    def _render(env: ReportEnvelope, spec: Optional[ReportSpec], audience: str) -> str:
        sections = spec.sections if (spec and spec.sections) else DEFAULT_SECTIONS
        title = (spec.title if spec and spec.title else None) or f"{env.identity.tool} ?
video report"
        if audience == "customer":
            title += " (summary)"
        parts = [f"# {title}", ""]
        for sec in sections:
            if sec.audience not in (audience, "both"):
                continue
            body = _build_section(env, sec, audience, spec)
            parts.append(f"## {sec.title}")
            parts.append("")
            parts.append(body)
            parts.append("")
        return "\n".join(parts).rstrip() + "\n"


    def render_technical(env: ReportEnvelope, spec: Optional[ReportSpec] = None) -> str:
        return _render(env, spec, "technical")


    def render_customer(env: ReportEnvelope, spec: Optional[ReportSpec] = None) -> str:
        return _render(env, spec, "customer")

```

27. user (2026-07-02T02:49:45.335Z)

```
C:\Users\avidu\Projects\khelsutra-guru\sports-obsreport\README.md:11:(`pydantic`, `Jinja2`,
`PyYAML`) so it is safe for commercial use.
C:\Users\avidu\Projects\khelsutra-guru\sports-obsreport\pyproject.toml:15:    "jinja2>=3.1.0",
```

28. user (2026-07-02T02:49:48.225Z)

```
.................................................................... [ 97%]
..                                                                   [100%]
74 passed in 0.43s
```

29. user (2026-07-02T02:49:48.663Z)

```
ruff 0.15.16
4 |
5 | from obsreport import Flag, Identity, ReportEnvelope, Stage, load_spec, run_spec
  |                       ^^^^
6 | from obsreport.rules import MISSING, apply_rules, eval_clause, get_path, to_rule_context
7 | from obsreport.spec import Clause
  |
help: Remove unused import: `obsreport.Flag`

Found 3 errors.
[*] 2 fixable with the `--fix` option (1 hidden fix can be enabled with the `--unsafe-fixes`
option).
===
tests/test_customer_message.py:2
tests/test_diff.py:2
tests/test_envelope.py:8
tests/test_io.py:7
tests/test_recorder.py:6
tests/test_render_json.py:3
tests/test_render_markdown.py:7
tests/test_rules.py:9
tests/test_schema.py:1
tests/test_spec.py:5
```

30. user (2026-07-02T02:49:57.863Z)

```
Exit code 1
src\obsreport\render\markdown.py:14:24: F401 [*] `..envelope.Flag` imported but unused
tests\test_recorder.py:38:31: F841 Local variable `s` is assigned to but never used
tests\test_rules.py:5:23: F401 [*] `obsreport.Flag` imported but unused
Found 3 errors.
[*] 2 fixable with the `--fix` option (1 hidden fix can be enabled with the `--unsafe-fixes`
option).
```

31. user (2026-07-02T02:49:59.878Z)

```
tests/test_reporting.py:80:# (see test_reporting_harvest.py which uses pytest.importorskip).
Those tests
tests/test_reporting_harvest.py:11:pytest.importorskip("obsreport")
tests/test_reporting_spec_faq.py:14:pytest.importorskip("obsreport")
4:and run metadata, populate an `obsreport.RunRecorder` with the raw signals. The
8:Imported only after `backend.reporting` confirms obsreport is installed, so it may
9:import obsreport freely.
19:    import obsreport  # soft dep; only needed for RunRecorder / load_spec at runtime
21:    obsreport = None  # type: ignore[assignment]
27:def load_spec() -> "obsreport.ReportSpec":
28:    return obsreport.load_spec(_SPEC_PATH)
83:def _validation_stage(rec: "obsreport.RunRecorder", val_results: List[Any]) -> None:
98:    rec: "obsreport.RunRecorder",
146:    rec: "obsreport.RunRecorder",
182:    rec: "obsreport.RunRecorder",
217:    spec: Optional["obsreport.ReportSpec"] = None,
```

```
220:) -> "obsreport.RunRecorder":
222:    rec = obsreport.RunRecorder(
```

32. assistant (2026-07-02T02:50:46.341Z)

Let me check the collector's CI usage of the `report` extra to gauge reachability, and verify a couple of README claims.

33. user (2026-07-02T02:50:48.157Z)

ci.yml

34. user (2026-07-02T02:50:49.552Z)

```
2026-06-07 13:17:39 +0530 1f72c1c feat: configurable highlights item_noun (v0.1.3)
2026-06-07 13:17:39 +0530 1f72c1c feat: configurable highlights item_noun (v0.1.3)
2026-06-07 13:14:16 +0530 2f5a214 feat: spec-level customer_verdict override (v0.1.2)
2026-06-07 13:03:41 +0530 cba67f2 feat: customer_message ? curated customer-facing flag text
(v0.1.1)
2026-06-07 12:51:20 +0530 7c40654 fix: correct setuptools build-backend (underscore) so
editable/pip install works
2026-06-07 12:50:24 +0530 d3e2a51 C4: RunRecorder + stable-diff + JSON Schema contract
2026-06-07 12:47:42 +0530 911adf7 C3: spec-driven markdown renderers (technical + customer) +
safe IO
2026-06-07 12:43:23 +0530 918a728 C2: report config language ? spec loader + restricted rule
engine
2026-06-07 12:39:29 +0530 687b52e C1: scaffold obsreport ? typed envelope + deterministic JSON
renderer
```

35. user (2026-07-02T02:50:58.189Z)

```
18:          python -m pip install --upgrade pip
19:          pip install -e ".[dev]"
```

36. user (2026-07-02T02:52:10.089Z)

Structured output provided successfully